

Software requirements Specification

for

**Retail Inventory and Online Order
Management System**

**Prepared by Group 16
July 27 2026**

Revision History

Version	Date	Reason

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 Product Scope
 - 1.5 References
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.2.1 User Operations
 - 2.2.2 Inventory and Admin Operations
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 User Documentation
 - 2.7 Assumptions and Dependencies
- 3. External Interface Requirements
 - 3.1 User Interfaces
 - 3.1.1 Logical Characteristics
 - 3.1.2 Software Components
 - 3.2 Hardware Interfaces
 - 3.2.1 Logical Characteristics
 - 3.2.2 Hardware Specifications
 - 3.3 Software Interfaces
 - 3.4 Communication Interfaces
- 4. System Features
 - 4.1 Product Browsing and Variant Management
 - 4.1.1 Description and Priority
 - 4.1.2 Stimulus/Response Sequences

- 4.1.3 Functional Requirements
- 4.2 Customer Registration and Authentication
 - 4.2.1 Description and Priority
 - 4.2.2 Stimulus/Response Sequences
 - 4.2.3 Functional Requirements
- 4.3 Shopping Cart and Order Placement
 - 4.3.1 Description and Priority
 - 4.3.2 Stimulus/Response Sequences
 - 4.3.3 Functional Requirements
- 4.4 Inventory Tracking
 - 4.4.1 Description and Priority
 - 4.4.2 Stimulus/Response Sequences
 - 4.4.3 Functional Requirements
- 4.5 Business Reports and Analytics
 - 4.5.1 Description and Priority
 - 4.5.2 Stimulus/Response Sequences
 - 4.5.3 Functional Requirements
- 4.6 Administrator Tools
 - 4.6.1 Description and Priority
 - 4.6.2 Stimulus/Response Sequences
 - 4.6.3 Functional Requirements
- 4.7 Payment Processing
 - 4.7.1 Description and Priority
 - 4.7.2 Stimulus/Response Sequences
 - 4.7.3 Functional Requirements
- 4.8 AI Chat Assistant
 - 4.8.1 Description and Priority
 - 4.8.2 Stimulus/Response Sequences
 - 4.8.3 Functional Requirements
- 5. Non-Functional Requirements
 - 5.1 System Reliability
 - 5.2 Performance and Responsiveness
 - 5.3 Availability and Operational Continuity
 - 5.4 Security and Access Control
 - 5.5 Scalability
 - 5.6 Maintainability and Code Quality
 - 5.7 Atomicity and Data Consistency
- 6. Additional Requirements
 - 6.1 Regulatory and Legal Compliance
 - 6.2 Localization and Accessibility
 - 6.3 Infrastructure, Backup and Disaster Recovery
 - 6.4 Reporting and Analytics
 - 6.5 System Auditing and Compliance Logging
- 7. Business Rules

Appendix A: Glossary
Appendix B: Analysis Models
Appendix C: To Be Determined List
Appendix D: Team Members

1. Introduction

1.1 Purpose

The purpose of this project is to design and implement a web-based **Retail Inventory and Online Order Management System** for BrightBuy. The system is intended to provide customers with a convenient online shopping experience by allowing them to browse products, select product variants, add items to a shopping cart, and complete purchases using available payment and delivery options. It also enables administrators to manage products, monitor inventory levels, process customer orders, and generate business reports. The overall goal is to improve inventory accuracy, streamline order processing, and support efficient retail operations through a reliable database-driven solution.

1.2 Document Conventions

This document follows the IEEE Software Requirements Specification (SRS) guidelines. It uses a consistent structure, numbering system, and formatting style throughout the document to improve readability. Tables, figures, and technical terms are presented in a clear and uniform manner.

The technical specifications within this document utilize the following nomenclature and formatting standards:

Standard Convention	Definition and Application
Shall	Indicates a mandatory, verifiable, and testable system requirement.
Should	Denotes a recommended practice that is not strictly compulsory.
May	Represents an optional feature or permissible capability.

Standard Convention	Definition and Application
REQ-X.X	Unique identifier for Functional Requirements (refer to Section 4).
NFR-X.X	Unique identifier for Non-Functional Requirements (refer to Section 5).
BR-XX	Unique identifier for Business Rules (refer to Section 7).
RPT-XX	Unique identifier for Report Specifications (refer to Section 8).

1.3 Intended Audience and Reading Suggestions

This document is intended for the stakeholders involved in the development, implementation, and maintenance of the BrightBuy Retail Inventory and Online Order Management System.

- **Project Managers** – Review the project objectives, scope, and overall requirements.
- **Software Developers** – Understand the system features, database structure, and implementation requirements .
- **Database Administrators (DBAs)** – Review the database design, relationships, constraints, and inventory management processes.
- **Testers (QA Engineers)** – Use the functional and non-functional requirements to develop and execute test cases.
- **Administrators** – Understand product, inventory, order, and report management functions.
- **Customers** – Refer to features related to product browsing, shopping cart, order placement, and order tracking.

1.4 Product Scope

The **BrightBuy system** is a web-based online retail platform designed to help customers and administrators manage shopping and inventory activities.

The system allows customers to:

- Browse available products and different product variants.
- Create accounts and securely log in.
- Add products to carts and place orders.
- Complete payments through supported payment methods.
- Select suitable delivery options.

The system also provides inventory management features to:

- Maintain accurate, real-time stock information.
- Validate product availability before confirming orders.
- Support administrators in monitoring and controlling stock levels.

Additionally, the system includes reporting features that help analyze sales performance and support business decisions.

The first release of the system has a limited scope:

- Delivery services are initially available only within **Texas**.
- The system supports limited product categories, such as **electronic gadgets and toys**.

1.5 References

1. IEEE Computer Society, *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*, 1998. Available:
<http://www.math.uaa.alaska.edu/~afkjm/cs401/IEEE830.pdf>
2. Perforce Software, *How to Write a Software Requirements Specification (SRS) Document*. Available:

<https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document>

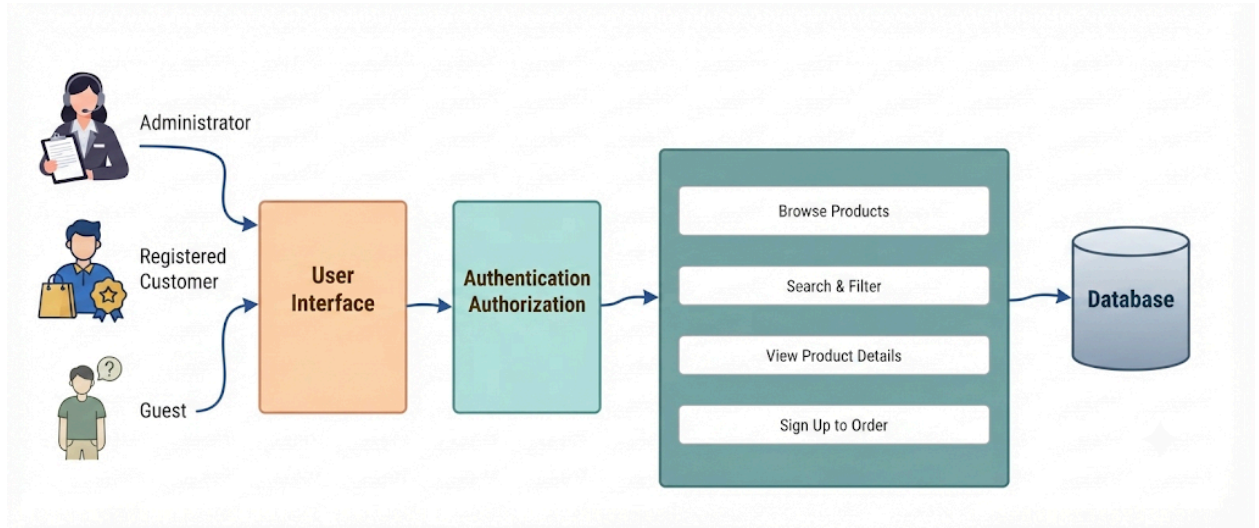
3. Relevant Software, *Your 2025 Guide to Writing a Software Requirements Specification (SRS) Document*. Available:

<https://relevant.software/blog/software-requirements-specification-srs-document/>

2. Overall Description

2.1 Product Perspective

The BrightBuy Retail Inventory and Online Order Management System is a web-based platform designed to serve as the backbone of BrightBuy's e-commerce operations. It integrates user-facing interfaces with secure authentication, core business logic, and centralized data management. The system supports multiple user roles, including administrators, registered customers, and guest users, each interacting with specific features through a unified interface. The following diagram illustrates the high-level architecture of the system and the interaction between users, the application, and the database.



2.2 Product Functions

The main functions of the BrightBuy Retail Inventory and Online Order Management System are:

2.2.1 User Operations

- **User Registration and Login**
 - Users can create an account and log in securely.
 - Different users have different access permissions.
- **Browse Products**
 - Users can view products by category, brand, and available variants (such as size or color).
- **Search Products**
 - Users can search for products and filter them by category, price, or availability.
- **Shopping Cart**
 - Users can add, update, or remove products from their shopping cart.
- **Order and Checkout**
 - Users can place orders and complete the checkout process.
- **Delivery Options**
 - Users can choose a delivery method and view the estimated delivery time.
- **Payment**
 - Users can pay using a credit/debit card or Cash on Delivery (COD).

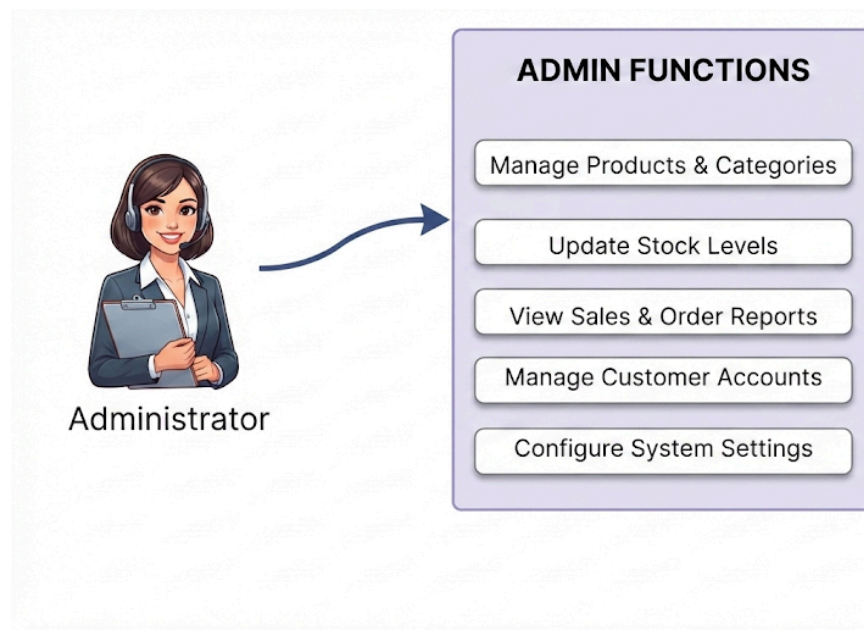
2.2.2 Inventory and Admin Operations

- **Stock Management**
 - The system checks product availability before confirming an order.
 - Stock is updated automatically after a successful order.
- **Admin Management**
 - Admins can add, edit, and delete products, categories, and product variants.
- **Reports**
 - Admins can view sales reports and monitor inventory and product performance.

2.3 User Classes and Characteristics

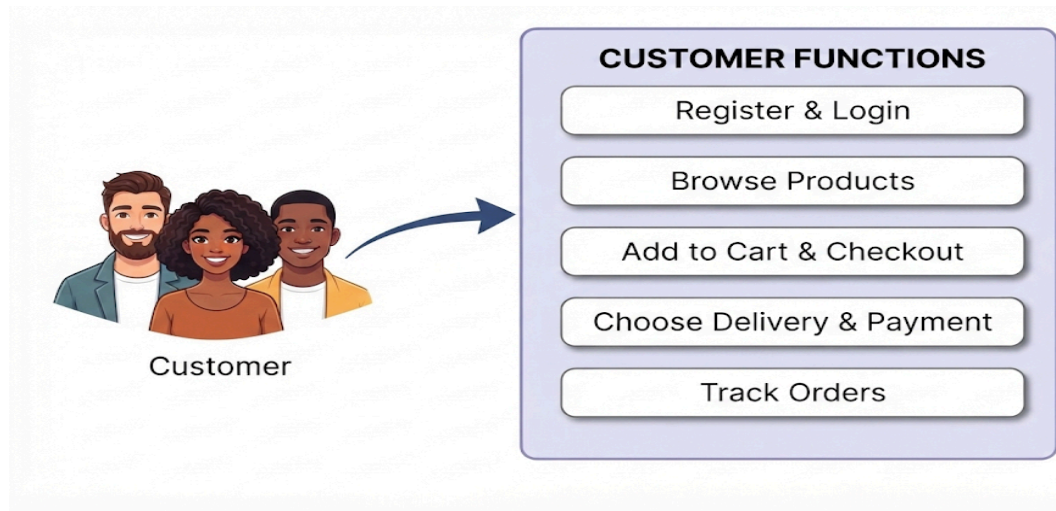
Admin Users

- Have basic knowledge of retail management or database systems.
- Have full administrative access to the system.



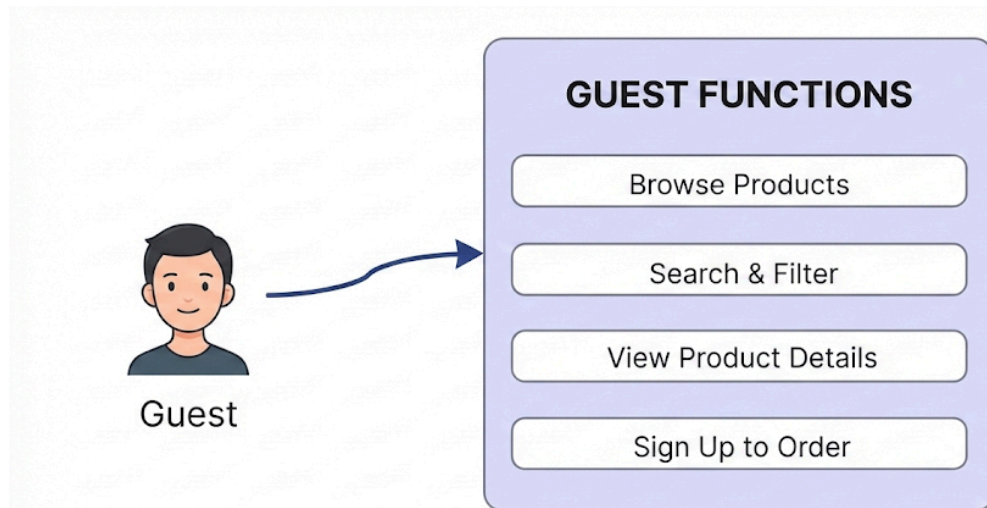
Registered Customers

- Can create an account and log in securely.
- Have full customer access to the system.
- Can browse products, manage their cart, place orders, make payments, and view order history.



Guest Users

- Do not need to register or log in.
- Have read-only access to the system.
- Can browse and search products but cannot place orders or access customer features.



2.4 Operating Environment

The Retail Inventory and Online Order Management System will operate in the following environment:

- **Platform**
 - A web-based application accessible through modern web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge.
- **Backend**
 - Developed using **Express.js**
 - Handles business logic, data processing, and RESTful APIs.
- **Frontend**
 - Developed using **HTML, CSS, and JavaScript**.
 - Uses **Next.js** for a responsive user interface.
- **Database**
 - Uses **PostgreSQL** to store and manage system data.
- **Hosting**
 - Can be deployed on cloud platforms (AWS, Azure, etc.) or on local servers.
- **Operating System**
 - Supports **Windows, Linux, and macOS**.
- **Security**
 - Uses **HTTPS** for secure communication.
 - Implements **Role-Based Access Control (RBAC)** for different user roles.
 - Maintains **audit logs** to track important system activities.

2.5 Design and Implementation Constraints

The development of the Retail Inventory and Online Order Management System must follow these constraints:

- **Secure Passwords**
 - User passwords must be securely encrypted using **bcrypt** or a similar hashing algorithm.
 - Passwords must never be stored in plain text.
- **Transaction Integrity**
 - The system must follow **ACID** properties to ensure reliable order processing and stock updates.
- **Atomic Operations**

- Orders and inventory updates must be completed as a single transaction to prevent data errors.
- **Unique Identifiers**
 - Each user must have a unique email address.
 - Each product must have a unique SKU (Stock Keeping Unit).
- **Payment Security**
 - Card payments must comply with **PCI-DSS** security standards.
- **Future Expansion**
 - The system should support future integration with external services such as delivery or logistics APIs.
- **Technology Requirements**
 - The system must use the technologies specified in **Section 2.4**, unless changes are approved by stakeholders.

2.6 User Documentation

The following documentation will be provided with the **Retail Inventory and Online Order Management System** to support users, administrators, and maintenance teams:

- **User Manual**
A complete guide explaining system features, user functions, and navigation steps. It will be available in **PDF format** and as an **online version within the system**.
- **Quick Start Guide**
A short guide for customers and administrators that explains the basic steps to start using the system.
- **Integrated Online Help**
The system will provide built-in help features such as **tooltips, user guidance, and FAQs** to assist users while using the application.
- **Administrator Training Guide**
A separate guide for administrators covering **system settings, product management, category management, reports, and user access control**.
- **Technical Documentation**
Technical documents will be provided for developers and maintenance teams, including:
 - **System Architecture Document** – Explains the system design, components, and technologies used.
 - **Entity-Relationship (ER) Diagram** – Shows the database structure and relationships between entities.

2.7 Assumptions and Dependencies

The following assumptions and dependencies may affect the development, deployment, and operation of the **Retail Inventory and Online Order Management System**.

Assumptions

- Users will access the system through modern web browsers such as **Chrome, Firefox, or Edge** with a stable internet connection.
- The warehouse system will regularly synchronize inventory data with the retail management system.
- Initial product information, stock details, variants, and SKUs will be manually entered or imported during system setup.
- Delivery services will be available within the supported delivery areas during the initial release.
- Card payments will be processed through a **third-party payment gateway**, and sensitive payment information will not be stored internally.

Dependencies

- The system depends on the availability and reliability of the **third-party payment gateway** for successful payment processing.
- Accurate inventory management depends on regular synchronization with the warehouse system.
- Future delivery tracking or logistics automation will require additional integration with external services.
- The backend system depends on selected technologies and frameworks such as **Node.js, Spring Boot, MySQL** for proper operation.

2.8 Database Design Requirements

As this initiative focuses extensively on architectural integrity, the development team must adhere to the following database-specific constraints and structural practices:

- **Primary Keys**
 - Each table structure shall implement a primary key to ensure the unique identification of every individual record.
 - Implementation examples include identifiers for users, products, orders, variants, and categories.

- **Foreign Keys**
 - The schema must define foreign key constraints to preserve referential integrity across related data entities, such as linking order line items to parent orders and specific product variations.

- **Indexing Strategies**
 - To optimize system performance, indexes shall be established on high-traffic columns, including:
 - Product nomenclature and grouping (for catalog filtering).
 - Transaction timestamps (to support analytical reporting).
 - User credentials (for rapid authentication lookups).
 - Stock Keeping Units (for real-time inventory management).

- **Stored Procedures**

- The backend shall utilize stored procedures to manage sophisticated logic, such as:
 - The order placement pipeline, encompassing stock validation, record creation, inventory deduction, and payment logging within a single atomic event.
 - Dynamic fulfillment and delivery time computations.
 - Aggregated quarterly revenue and sales summaries.

- **Database Functions**

- Reusable computational logic shall be implemented through database functions for:
 - Aggregating total purchase values from individual cart items.
 - Predicting arrival dates based on regional variables and warehouse status.
 - Summing order volumes categorized by product group.

- **Database Triggers**

- Triggers shall automate the enforcement of critical business rules:
 - Pre-insertion validation on line items to confirm sufficient warehouse availability.
 - Post-insertion automation to immediately adjust variant quantities after a sale.
 - Strict inventory guards to reject any update resulting in negative stock balances.

- **Normalization Standards**

- All relational schemata must be normalized to a minimum of Third Normal Form (3NF) to mitigate data redundancy and maintain consistency.

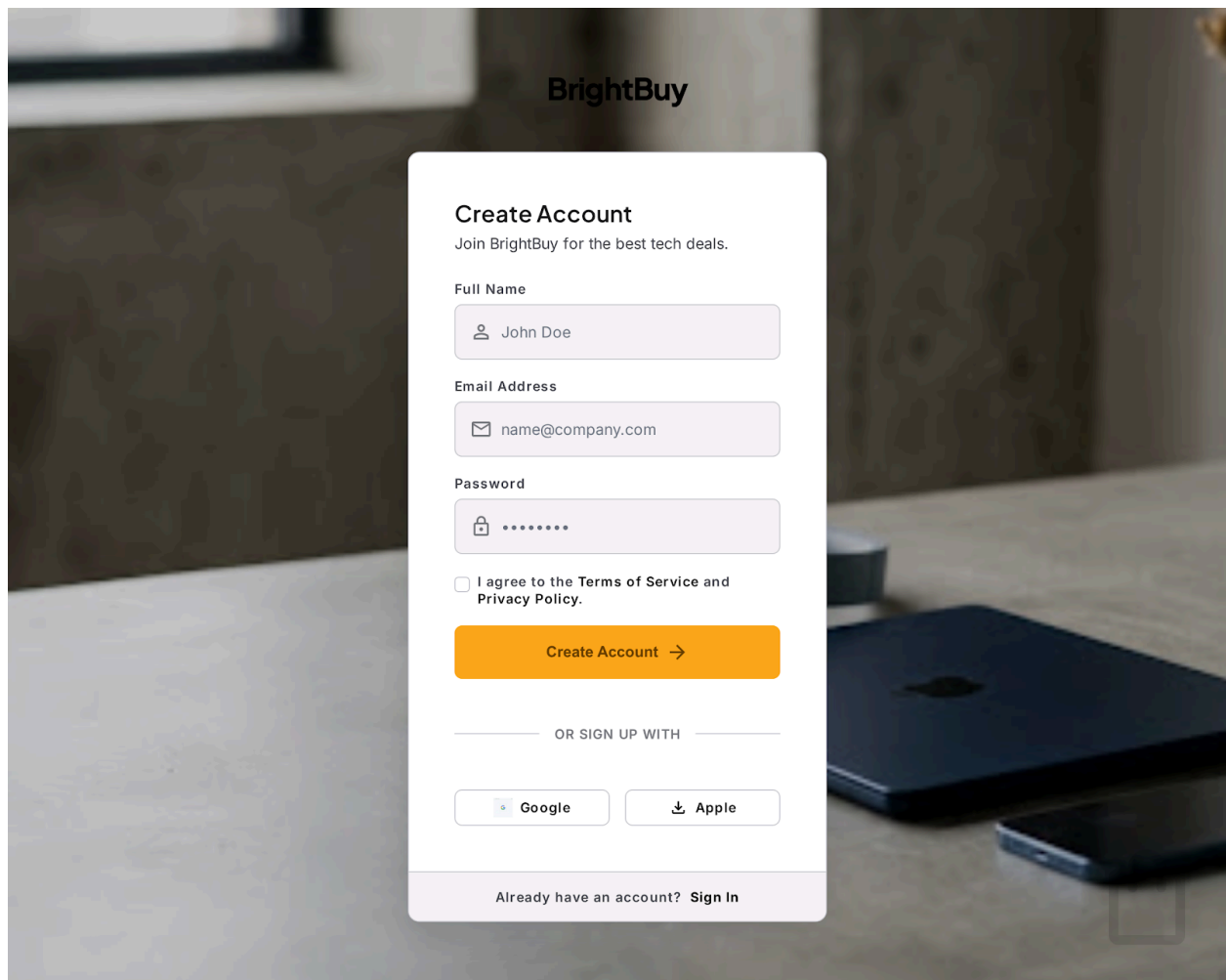
- **Initial Data Population**

- For validation and demonstration, the repository must be populated with a baseline of 40 products distributed across 10 distinct categories, inclusive of all relevant variants.

3. External Interface Requirements

3.1 User Interfaces

The system offers a straightforward, user-friendly experience for both shoppers and staff. Customers can easily explore products and place orders, while administrators can seamlessly track stock and manage fulfillment. Built on modern web standards, the design guarantees a consistent, accessible, and mobile friendly experience across all devices.



User Interface and Design Rules

1. Visual Style and Navigation

- **Modern Look:** The system uses Material Design principles to ensure the interface feels clean, uncluttered, and familiar to users.

- **Brand Identity:** The screens strictly follow BrightBuy's official color palette, built around a core theme of green and white with specific accent colors.
- **Easy Navigation:** The menus automatically adapt to the user's device. Desktop users will see a standard navigation bar across the top, while mobile users will get a space-saving hamburger menu.

2. Screen Size and Accessibility

- **Responsive Layout:** The platform must look and function perfectly across all devices, seamlessly adjusting to desktops, tablets, and smartphones.
- **Supported Sizes:** The design is built to scale smoothly from small mobile screens (320px minimum width) all the way up to large desktop monitors (1920px width).
- **Inclusive Design:** The system meets standard web accessibility guidelines to ensure it can be easily used by everyone, regardless of disability.

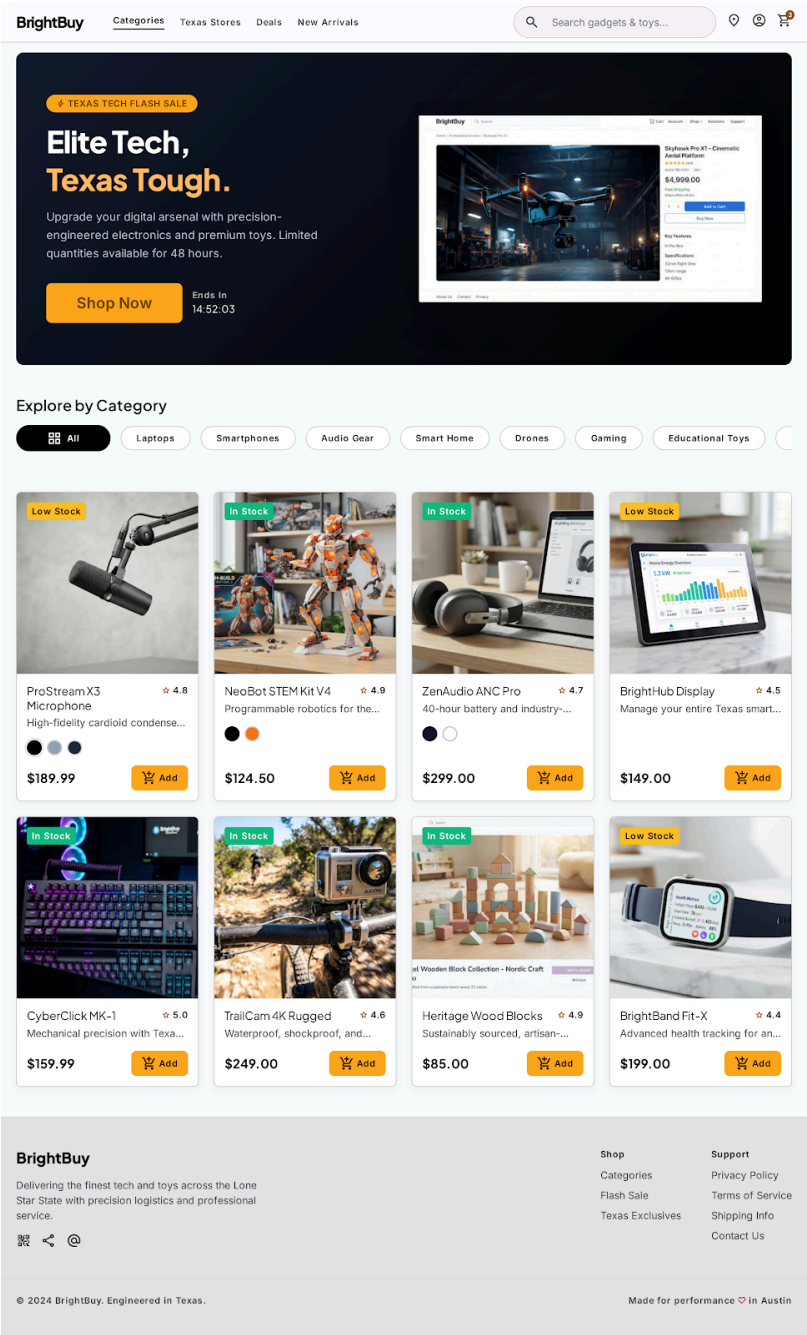
3. Standard Controls and Global Elements

- **Always-Visible Tools:** No matter what page the user is on, the top-right corner will always feature a search bar, an account login button, and a shopping cart showing the current number of items. A help link is permanently pinned to the footer.
- **Keyboard Shortcuts:** The system supports common, time-saving keyboard shortcuts, such as pressing **Ctrl + F** to search the current screen or **Esc** to close pop-up windows.
- **Helpful Error Messages:** When something goes wrong, the system provides plain-language alerts that explain exactly how to fix the issue, alongside a specific error code in case technical support is needed.

Software Components

1. Product Catalog & Search

Users can explore products by category using a filter sidebar. Each product is displayed in card format, with infinite scroll enabling seamless browsing across large catalogs.



2. Shopping Cart & Checkout

Customers can view, update, or remove items in their cart. The checkout process includes a structured form for delivery and payment details, followed by an order confirmation screen.

BrightBuy

CategoriesTexas StoresDealsNew Arrivals

Search gadgets...

📍🔄🛒

Your Cart

CyberTyper Pro v2

Variant: Midnight Blue / Linear Switches

-1+

Remove

\$189.00
~~\$249.00~~

AuraStream Wireless

Variant: Titanium Silver

-1+

Remove

\$299.00

Delivery Mode

Standard Delivery

ETA: Oct 24 - Oct 26

\$12.50

Store Pickup

Texas Warehouse #402

FREE

Payment Method

Credit / Debit Card

Card Number

MM/YY

CVV

Cash on Delivery

Order Summary

Subtotal (2 items)\$488.00

Shipping Fee\$12.50

Tax (Texas 8.25%)\$41.29

Total\$541.79

Your purchase is protected by BrightBuy Assurance for 12 months.

Place Order →

By placing your order, you agree to our Terms of Service.

Need assistance?

Chat with a logistics expert.

BrightBuy

© 2024 BrightBuy. Engineered in Texas.

Privacy Policy

Terms of Service

Shipping Info

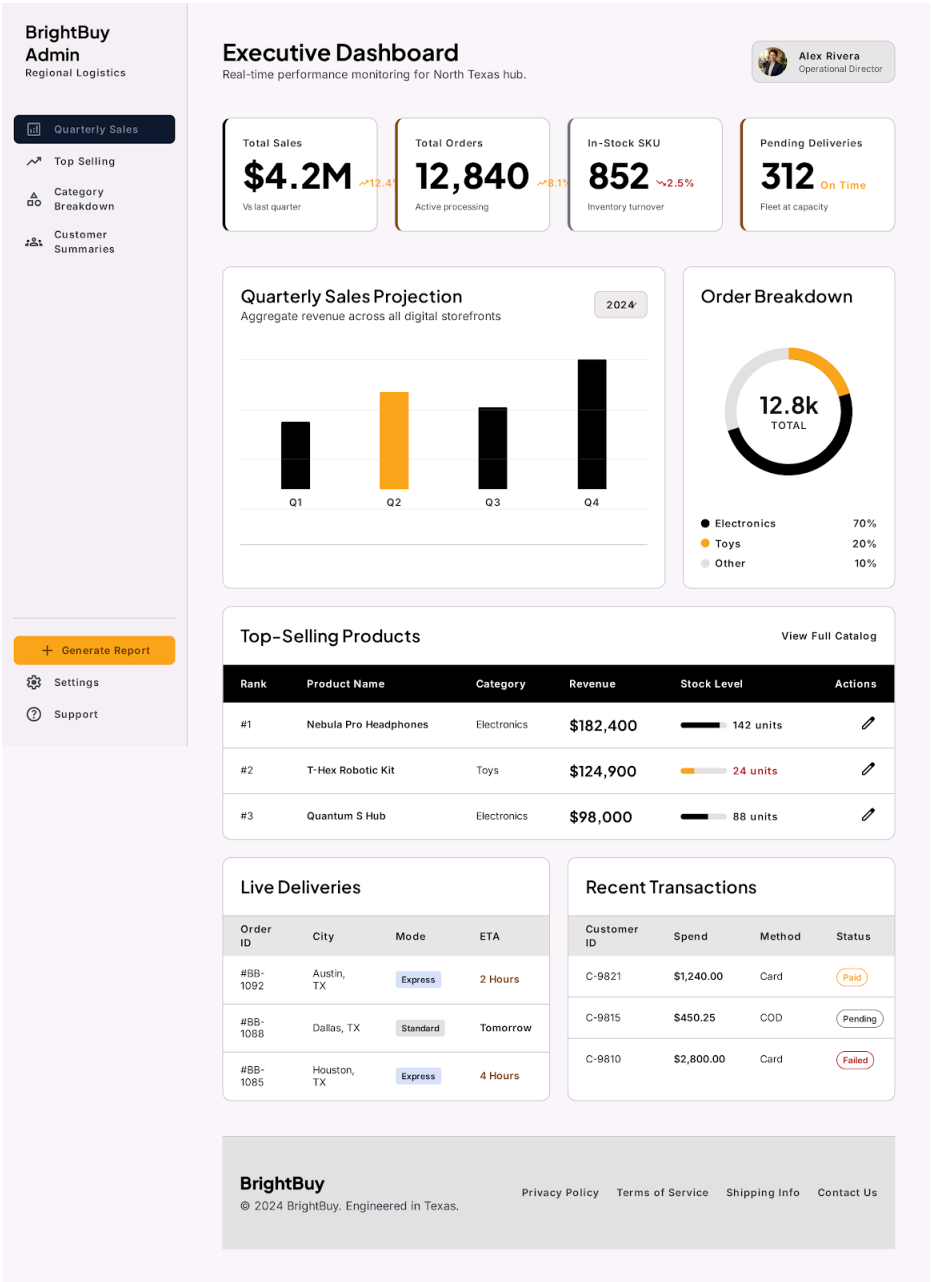
Contact Us

3. Order Management(Admin)

Admins can track and manage customer orders through an interactive order table. Key actions, such as marking orders as shipped, are available directly from the interface.

4. Inventory Management(Admin)

Administrators can monitor and update stock levels using an editable stock table. Bulk import/export options are available to streamline large inventory changes.



To meet IEEE standards, the system's interface must be thoroughly tested and clearly documented. Testing will focus on:

- **Compatibility:** Making sure the system is accessible and works smoothly across all devices and operating systems.
- **Functionality:** Checking that all buttons, features, and layout adjustments work exactly as expected.
- **Clarity:** Ensuring that error messages are helpful and consistent.

3.2 Hardware Interfaces

The system connects with several hardware devices to keep operations running smoothly everywhere from the warehouse backroom to the physical retail store and the customer's personal phone.

Logical Characteristics

1. Device Compatibility & Responsiveness

The system works seamlessly on desktop computers, laptops, tablets, and smartphones by automatically adjusting to fit any screen size. It is fully touch-friendly for mobile users and easily connects with physical store hardware like barcode scanners and receipt printers.

2. Control & Connective Features

To stay reliable in real-world conditions, the system includes:

- **Automatic Reconnection:** If the internet drops, the system saves data locally and automatically syncs it to the main server once the connection is restored.
- **Battery Saving:** On portable devices like store tablets, the software pauses idle background tasks to prevent battery drain.
- **Location Tracking:** The system can use device GPS to provide real-time shipment updates for deliveries.

3. Communication Protocols

The software talks to the hardware using standard, secure methods:

- **HTTPS & WebSockets:** Used to keep web transactions secure and push instant, real-time notifications to users.
- **Bluetooth & Wi-Fi:** Used to wirelessly connect physical peripherals like scanners and receipt printers.

Hardware Specifications

- **Processing Power:** The backend server requires an x86-64 processor to handle heavy order traffic. For the user side, any device with at least a dual-core 1.5GHz processor will run the system smoothly.
- **Storage:** The central database runs on high-speed SSDs for fast searches, while mobile devices use a small amount of local storage to keep the app working offline.
- **Inputs & Outputs:** The system accepts commands from touchscreens, standard keyboards, and barcode scanners, and outputs data to screens and receipt printers.
- **Operating Systems:** The backend server runs on either Linux or Windows. The customer and staff interfaces run on any standard web browser or mobile operating system.

Testing to IEEE Standards

To ensure the system meets IEEE standards, all software and hardware connections will be rigorously tested. This includes verifying that the communication protocols (like Bluetooth and Wi-Fi) connect properly, ensuring the interface responds quickly on all devices, and confirming that data moving between the hardware and software is perfectly accurate and secure.

3.3 Software Interfaces

- **Operating Systems:**
The system architecture is designed for deployment on Linux servers. End users and administrative personnel can interface with the application via any modern web browser across Windows, macOS, Android, and iOS, utilizing responsive layouts that dynamically optimize for mobile and tablet hardware.
- **Database Management System (DBMS):**
The platform utilizes **PostgreSQL** for the secure archival of all primary datasets, including product catalogs, user profiles, transaction histories, and AI conversation state. The database layer enforces strict relational constraints to ensure the precision of inventory and financial records during concurrent user interactions.
- **Backend Application:**
The central processing engine (governing cart logic, order fulfillment, and AI request orchestration) is a sophisticated API developed with **Express.js**. It facilitates stateless communication with the frontend via standard JSON objects, guaranteeing efficient and protected data transmission.
- **Frontend Application:**
The user-centric interface will be constructed using **Next.js** (React.js-based), providing a fluid and interactive experience. This framework leverages Server-Side Rendering (SSR) and Static Site Generation (SSG) to maximize performance and minimize page load durations.

- **Artificial Intelligence & Chat Integrations:**
The system integrates with Gemini or Grok APIs to deliver a natural-language assistant for discovery and support. The backend employs **LangGraph** to manage stateful dialogue workflows, ensuring that all AI-generated responses remain contextually accurate and relevant.
- **Security and Access Control:**
Authorization is handled via JSON Web Tokens (JWT) to implement stateless, Role-Based Access Control (RBAC). This logic isolates customer profiles from administrative tools, while session timeouts and HTTPS encryption provide additional layers of data protection.
- **Payment Gateway:**
Financial transactions are processed through **Lemon Squeezy**, which functions as the Merchant of Record (MoR) to manage global taxation and compliance. Secure tokenization ensures that sensitive cardholder data is never stored within the local system database.
- **Notification Services:**
The application integrates with **Resend** to automate the dispatch of order confirmations and shipping updates. This ensures that all outbound customer communications are well-formatted and maintain high deliverability standards.
- **Reporting Tools:**
To facilitate data-driven decisions, the reporting module generates comprehensive summaries of sales revenue, inventory performance, consumer behavior patterns, and AI engagement analytics for administrative review.

3.4 Communication Interfaces

The BrightBuy system uses secure, fast connections to keep all data safe:

- **Web Traffic:** All communication between the website and the server is strictly encrypted (HTTPS) to protect user data from being intercepted.
- **Internal Speed:** The core system components talk to each other instantly to guarantee real-time updates for inventory and orders.
- **Payment Gateways:** The software securely connects to external payment processors (like Stripe) using protected keys to safely process transactions and confirm payments.
- **Email Alerts (Future):** Order receipts and shipping updates will be sent securely to customers using trusted email providers.
- **Database Security:** The connection between the software and the database is heavily encrypted (SSL) and locked down to protect all customer and product records.

4. System Features

4.1 Catalog Browsing and Variant Selection

4.1.1 Description and Priority

High Priority – This functionality allows verified users and guest shoppers to navigate the product range. Catalog entries may feature diverse options like sizes or colors, which directly influence the individual SKU's pricing and current availability levels.

4.1.2 Stimulus/Response Sequences

- A user selects a product category → The platform dynamically displays all items categorized within that specific group.
- A shopper views a specific item → The system provides comprehensive details, including available variants, cost tiers, and warehouse stock status.
- A user performs a keyword search → The software processes the input and returns matching results from the inventory.

4.1.3 Functional Requirements

- **REQ-1.1:** The interface shall include navigation menus to filter the catalog by primary groups and secondary sub-groups.
- **REQ-1.2:** The system must clearly show the unique SKU, specific price, and real-time quantity for every product variation.
- **REQ-1.3:** The application shall feature a search engine capable of finding goods via keywords, descriptive tags, or stock status.
- **REQ-1.4:** Guest visitors shall have permission to view products, though the platform must restrict them from cart modifications or purchasing.

4.2 Customer Registration and Authentication

4.2.1 Description and Priority

High Priority – This component manages new user onboarding, identity verification, and role-based permissions. It employs industry-standard encryption to protect shopper data, and only verified accounts may proceed to the checkout phase.

4.2.2 Stimulus/Response Sequences

- A new shopper submits registration details -> The backend validates the input, hashes the password, and establishes a new profile.
- A returning member enters credentials -> The software authenticates the login and directs the user to their personal dashboard.
- A session reaches the inactivity limit -> The system terminates the session automatically to ensure account protection.

4.2.3 Functional Requirements

- **REQ-2.1:** Account creation shall require a verified email, complex password, and primary contact details from the user.
- **REQ-2.2:** The backend shall utilize bcrypt to hash all user passwords upon receipt, preventing the storage of readable credentials.
- **REQ-2.3:** The platform must manage secure user sessions through the implementation of JSON Web Tokens (JWT).
- **REQ-2.4:** Access to payment gateways and order finalization workflows shall be limited strictly to authenticated customers.

4.3 Shopping Cart and Order Processing

4.3.1 Description and Priority

High Priority – This essential module enables users to aggregate items and complete purchases. It includes real-time stock validation to prevent overselling and ensures transactional integrity within the database during order confirmation.

4.3.2 Stimulus/Response Sequences

- A shopper adds an item to their cart → The software updates the cart status and adjusts the preliminary total.
- A user initiates checkout → The backend verifies final stock → Upon approval, it collects shipping data → The system confirms the sale and updates inventory.

4.3.3 Functional Requirements

- **REQ-3.1:** The platform shall allow logged-in shoppers to add, remove, or change item quantities in their active shopping cart.
- **REQ-3.2:** The backend must perform an inventory audit for all cart items immediately before the payment stage begins.
- **REQ-3.3:** The database shall use atomic operations to link stock deduction with order creation, ensuring data consistency.
- **REQ-3.4:** The checkout system shall offer two fulfillment methods: Local Store Pickup or standard Home Delivery.
- **REQ-3.5:** The checkout pipeline must support at least two payment options: Credit/Debit Cards and Cash on Delivery.
- **REQ-3.6:** Expected delivery dates shall be calculated as follows:
 - Major Texas cities: 5 business days for items in stock.
 - Other Texas regions: 7 business days for items in stock.
 - Depleted stock: An additional 3 business days added to the estimated arrival time.

4.4 Real-Time Inventory Management

4.4.1 Description and Priority

High Priority – This system tracks the physical count of every item variant. It utilizes strict database constraints and automated triggers to ensure stock levels remain accurate and items are never sold beyond availability.

4.4.2 Stimulus/Response Sequences

- A payment is confirmed → The backend immediately deducts the purchased items from the available inventory records.

- An admin manually edits stock → The database applies the change and logs the modification for auditing purposes.
- Stock reaches zero → The storefront interface disables the "Add to Cart" functionality for that specific variant.

4.4.3 Functional Requirements

- **REQ-4.1:** The database shall maintain precise, persistent quantity tracking for every unique variant in the catalog.
- **REQ-4.2:** Stock deduction must be prohibited until the system receives a successful payment authorization signal.
- **REQ-4.3:** The system architecture must implement triggers to reject any transaction that would create a negative inventory balance.
- **REQ-4.4:** All order-related stock updates must be executed as atomic database events to prevent concurrency discrepancies.
- **REQ-4.5:** An immutable audit trail shall record all manual stock overrides, including timestamps, values changed, and the administrator responsible.

4.5 Business Reports and Analytics

4.5.1 Description and Priority

Medium Priority – This engine provides management with strategic data on store performance. It generates reports on revenue trends, product popularity, category performance, and historical customer activity.

4.5.2 Stimulus/Response Sequences

- An admin sets report criteria → The backend compiles the data and displays a summary on the dashboard.
- A user chooses to download → The system exports the reporting data into the selected file format.

4.5.3 Functional Requirements

- **REQ-5.1:** The platform shall generate aggregated sales revenue documents segmented by financial quarters.
- **REQ-5.2:** The system must identify and rank top-selling products based on customized timeframe parameters.
- **REQ-5.3:** Reporting tools shall provide summaries of total order volumes categorized by product group.
- **REQ-5.4:** The analytics interface must project estimated fulfillment dates for all currently pending orders.
- **REQ-5.5:** The backend shall compile activity profiles for individual customers, including order history and payment records.
- **REQ-5.6:** All analytical data shall be exportable into standard business formats, including CSV, PDF, and Excel.

4.6 Administrator Management Tools

4.6.1 Description and Priority

Medium Priority – This module provides a secure dashboard where authorized staff can manage the product catalog, modify stock details, and organize categories.

4.6.2 Stimulus/Response Sequences

- A user logs in with staff privileges → The platform grants access to the administrative control panel.
- An admin creates a product entry → The backend validates the data and assigns a unique SKU to the item.
- A manager updates item details → The system immediately applies the changes to the live web catalog.

4.6.3 Functional Requirements

- **REQ-6.1:** The dashboard shall provide tools to create, edit, or delete individual products and entire category structures.
- **REQ-6.2:** The system must enforce uniqueness for all SKUs to prevent identification errors within the product catalog.
- **REQ-6.3:** The database shall maintain a log of all catalog updates, preserving a history of administrative changes.

- **REQ-6.4:** The platform shall include bulk processing utilities for importing and exporting large-scale catalog data.

4.7 Secure Payment Processing

4.7.1 Description and Priority

High Priority – This component handles the financial requirements of the order process. It ensures sales are only finalized upon successful payment and links every transaction record to the corresponding order.

4.7.2 Stimulus/Response Sequences

- A customer provides payment details → The backend securely communicates with the processor → On approval, the sale is confirmed → On rejection, the process stops and alerts the user.

4.7.3 Functional Requirements

- **REQ-7.1:** The interface must permit shoppers to choose between direct card payment or Cash on Delivery.
- **REQ-7.2:** The database shall permanently link the payment method and transaction ID to the final customer order.
- **REQ-7.3:** The system must require a positive confirmation from the payment gateway before updating an order to "Confirmed" status.
- **REQ-7.4:** Reporting tools shall track and display statistics regarding the most frequently used payment methodologies.
- **REQ-7.5:** No physical inventory shall be deducted or reserved if a payment transaction is abandoned or declined.

4.8 AI Chat Assistant

4.8.1 Description and Priority

Medium Priority – This interface provides an automated assistant to handle common customer questions and shopping help through a persistent on-screen widget.

4.8.2 Stimulus/Response Sequences

- A user opens the chat icon → The interface expands and displays a welcoming prompt.
- A user types a question → The AI engine generates a relevant answer or provides helpful navigation links.
- The user closes the chat → The system saves the conversation record for future AI training and improvement.

4.8.3 Functional Requirements

- **REQ-8.1:** The website design shall feature a floating chat component accessible from all main pages.
- **REQ-8.2:** The chat module shall use auto-suggestions to help shoppers create support requests more quickly.
- **REQ-8.3:** Automated support shall be available to both registered members and anonymous visitors.
- **REQ-8.4:** The backend must log chat transcripts securely to support continuous improvement of the AI logic.
- **REQ-8.5:** The chat widget must be responsive, working effectively on both desktop monitors and mobile devices.

5. Non-Functional Requirements

5.1 System Reliability

- **NFR-REL-01 — Data Integrity:** The platform must guarantee complete data accuracy across inventory levels, order state tracking, and customer billing information.
- **NFR-REL-02 — Transactional Reliability:** All multi-step database interactions (such as simultaneous order processing and stock deduction) must abide by ACID properties to prevent partial data writes during system disruptions or network failures.
- **NFR-REL-03 — Error Recovery:** In the event of an unexpected hardware or network fault, the application must automatically rollback incomplete transactions to maintain data consistency.

5.2 Performance and Responsiveness

- **NFR-PERF-01 — Response Time:** User-facing dynamic pages (e.g., product catalog browsing, cart updates, checkout workflow) must render in under **2 seconds** under normal load conditions.
- **NFR-PERF-02 — Query Efficiency:** Search queries, filtering actions, and analytical reporting features (such as top-selling items or sales summaries) must complete within **3 seconds**, even as the product catalog and transaction history grow large.
- **NFR-PERF-03 — Throughput:** The system API must support up to **100 concurrent requests per second** without noticeable latency spikes.

5.3 Availability and Operational Continuity

- **NFR-AVAIL-01 — Uptime Standard:** The web application must maintain **99.5% operational uptime** (\$24/7\$ availability), excluding scheduled maintenance windows.
- **NFR-AVAIL-02 — Fault Tolerance:** Critical paths, such as product browsing and cart management, should remain accessible even if secondary services (like automated email notifications) experience temporary downtime

5.4 Security and Access Control

- **NFR-SEC-01 — Data Protection:** Confidential user records, billing addresses, and payment tokens must be encrypted both in transit (using TLS/HTTPS) and at rest (using AES-256)
- **NFR-SEC-02 — Role-Based Authorization (RBAC):** Access privileges are strictly segregated:

Unauthenticated Guests: Browsing catalog only.

Authenticated Customers: Cart management, order execution, personal profile viewing.

System Administrators & Inventory Staff: Catalog management, stock adjustments, and sales analytics access.

5.5 Scalability

- **NFR-SCALE-01 — Horizontal & Vertical Expansion:** The system backend and database architecture must accommodate growing data volumes (products, user records, order history) without architectural refactoring.

- **NFR-SCALE-02 — Traffic Spikes:** The infrastructure must support dynamic scaling to handle high transaction volumes during promotional sales or holiday peak periods.

5.6 Maintainability and Code Quality

- **NFR-MAIN-01 — Modular Architecture:** The codebase must adhere to a layered architectural pattern (e.g., MVC or microservices/REST pattern) to facilitate independent component updates.
- **NFR-MAIN-02 — Database Normalization:** Relational database schemata must be normalized to minimum 3NF to eliminate data redundancy and preserve maintainability.
- **NFR-MAIN-03 — Extensibility:** Interface designs must support seamless future feature additions (such as multi-currency support, product reviews, or return/refund workflows).

5.7 Atomicity and Data Consistency

- **NFR-ATOM-01 — Atomic Order-Stock Transactions:** Order creation and inventory deduction must execute as a single, atomic database operation.
- **NFR-ATOM-02 — Consistency Guarantee:** If a checkout transaction or payment authorization fails midway, all state changes must be completely reverted, leaving stock balances unchanged.

6. Additional Requirements

6.1 Regulatory & Legal Compliance

- **Data Privacy Regulations:** The platform must comply with local consumer data protection regulations (including Texas state privacy laws) regarding user data retention and user account deletion requests.
- **Payment Security Compliance:** All credit/debit card handling must conform to **PCI-DSS Level 1** guidelines. Direct credit card processing should be offloaded to third-party secure gateways (e.g., Stripe) to prevent local handling of raw card details
- **Consumer Protection:** Product descriptions, pricing structures, and refund policy terms must comply with transparent commerce regulations (such as the Texas Deceptive Trade Practices Act).

6.2 Localization & Accessibility

- **Regional Optimization:** The default system time zone must be set to **Central Standard Time (CST/CDT)** for transaction logging and reporting accuracy.
- **Delivery Calculation:** Estimated shipping times shall be computed as follows:
 - Main cities in Texas: 5 business days if the item is in stock.
 - Other Texas cities: 7 business days if the item is in stock.
 - Out-of-stock items: Add 3 additional business days to the above estimates.
 - These estimates shall be displayed clearly during checkout and in delivery time reports.
- **Web Accessibility:** Interface designs must follow **WCAG 2.1 Level AA standards**, including proper color contrast ratios, screen-reader compatibility, keyboard navigation support, and dynamic font scaling.

6.3 Infrastructure, Backup & Disaster Recovery

- **Stock Oversell Prevention:** The core database engine must utilize optimistic/pessimistic locking mechanisms during checkout to prevent race conditions and overselling.
- **Backup Strategy:** Automated full database backups must execute daily, accompanied by transaction log backups every 4 hours to minimize potential data loss.
- **Disaster Recovery:** The application must maintain a **Recovery Point Objective (RPO) of < 1 hour** and a **Recovery Time Objective (RTO) of < 4 hours** in event of a critical server failure

6.4 Reporting & Analytics

- **Business Intelligence:** The system must generate configurable reports on demand, covering metrics such as top-performing products, category revenue breakdowns, customer purchase frequency, and return rates.
- **Export Capabilities:** Analysts and admins must be able to export reporting data into universally supported formats, including **CSV, Excel, and formatted PDF files**.

6.5 System Auditing & Compliance Logging

- **Audit Trail:** The backend must maintain immutable audit logs recording high-value administrative actions (e.g., manual stock overrides, user permission adjustments, order status updates).
- **Log Retention:** System and audit logs must be securely archived for a minimum of 12 months for compliance and forensic analysis

7. Business Rules

The following directives define the operational logic and policy constraints of the BrightBuy ecosystem. While distinct from direct technical requirements, the functional and non-functional specifications detailed previously are engineered to enforce these principles across the platform.

- **BR-01:** Fulfillment capabilities are restricted to the geography of Texas during the primary launch phase.
- **BR-02:** Shipments to metropolitan Texas zones carry a 5-business-day arrival estimate for available inventory.
- **BR-03:** Orders destined for regional Texas areas are estimated at 7 business days for items currently in stock.
- **BR-04:** Depleted stock levels necessitate a 3-business-day extension to the standard shipping timeline.
- **BR-05:** The platform accepts financial settlement via standard Credit/Debit Cards or the Cash on Delivery (COD) method.
- **BR-06:** Order finalization and stock reservation are contingent upon verified payment authorization; inventory remains unallocated until this event.
- **BR-07:** The system enforces absolute SKU uniqueness for every product variation within the global catalog.

- **BR-08:** User profile establishment requires a validated, unique email address for identity management.
- **BR-09:** Credential security mandates the use of bcrypt hashing; the persistence of unencrypted passwords is strictly prohibited.
- **BR-10:** Anonymous visitors possess read-only privileges and are restricted from cart manipulation or transaction initiation.
- **BR-11:** Architectural guards must prevent negative inventory balances, ensuring data consistency during concurrent user sessions.
- **BR-12:** Initial inventory is limited to specific retail groups, primarily encompassing electronic devices and toys.
- **BR-13:** Fiscal events must be recorded with a permanent audit trail, including specific reference IDs and precise time-stamping.
- **BR-14:** Manual inventory overrides require formal documentation, capturing the delta values and the identity of the modifying administrator.

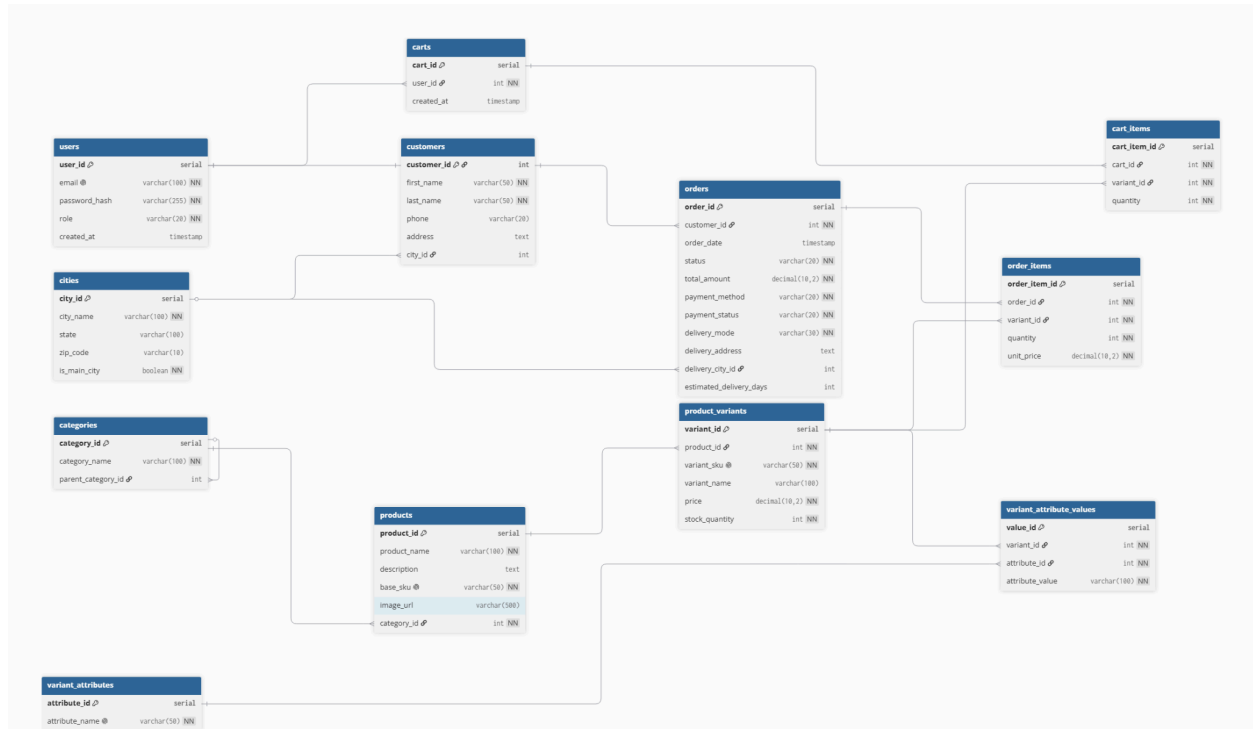
Appendix A: Glossary

- **ACID** – An acronym for Atomicity, Consistency, Isolation, and Durability, representing the fundamental standards that guarantee database transactions remain reliable and free from corruption.
- **API** – Application Programming Interface; a structured set of protocols that facilitate communication and data transfer between independent software components.
- **bcrypt** – A robust cryptographic hashing algorithm utilized to secure user passwords through irreversible encryption, effectively defending against brute-force intrusion attempts.
- **COD** – Cash on Delivery; a transaction method where the purchaser provides payment at the point of physical delivery rather than during the digital checkout phase.
- **DBMS** – Database Management System; software platforms like MySQL or PostgreSQL designed to organize, manage, and retrieve structured information from a data repository.
- **ER Diagram** – Entity-Relationship Diagram; a visual schematic illustrating the database architecture, including tables and the relational constraints that link them.
- **HTTPS** – HyperText Transfer Protocol Secure; a network protocol that employs TLS encryption to safeguard web communication from unauthorized interception or manipulation.
- **IEEE** – The Institute of Electrical and Electronics Engineers; a global professional association dedicated to the advancement of technology and the publication of technical standards.

- **JWT** – JSON Web Token; a secure, digitally signed format for transmitting identity and authorization data between client applications and server backends.
- **MySQL** – A popular open-source relational database management system often utilized for managing backend data for large-scale web applications.
- **PCI-DSS** – Payment Card Industry Data Security Standard; a mandatory safety framework ensuring that businesses handle, process, and store cardholder information securely.
- **RBAC** – Role-Based Access Control; a security method that restricts system access based on assigned user roles, such as administrator, customer, or guest.
- **REST/RESTful** – Representational State Transfer; an architectural design for networked systems that leverages standard HTTP methods to manage stateless data exchange.
- **RPO** – Recovery Point Objective; the maximum age of files that must be recovered from backup storage for operations to resume after a system failure.
- **RTO** – Recovery Time Objective; the targeted duration of time within which a business process must be restored following a service interruption.
- **SKU** – Stock Keeping Unit; a unique identifier assigned to each product variant to facilitate efficient inventory tracking and warehouse management.
- **SRS** – Software Requirements Specification; a comprehensive document detailing the functional requirements, constraints, and performance goals of a development project.
- **SSL/TLS** – Secure Sockets Layer and Transport Layer Security; cryptographic protocols designed to provide secure, encrypted communication over a computer network.
- **WCAG** – Web Content Accessibility Guidelines; a set of international standards that ensure web content is accessible to and usable by individuals with disabilities.

Appendix B: Analysis Models

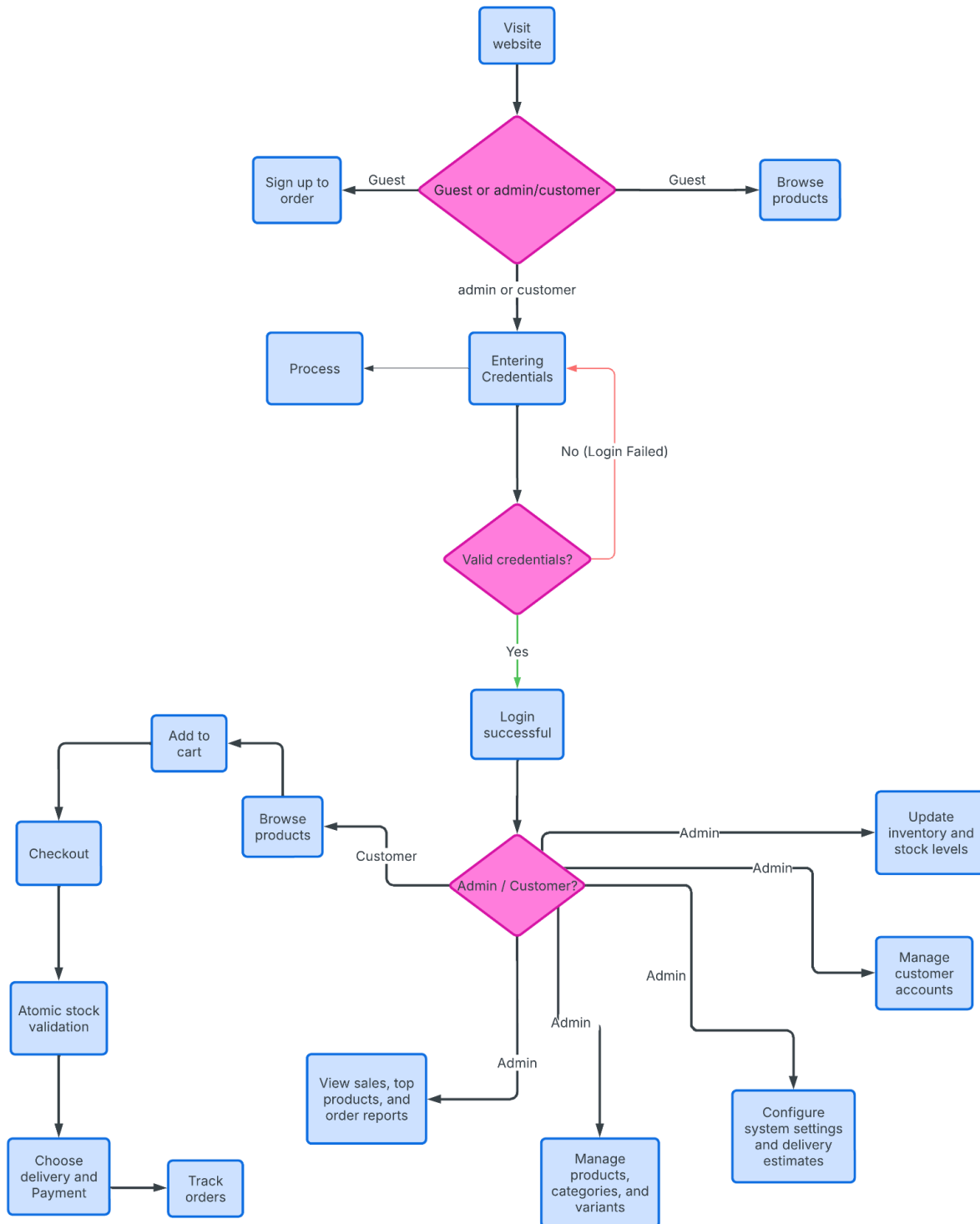
B.1 Entity-Relationship Diagram



The ER diagram above illustrates the database structure of the BrightBuy system, showing the primary entities and their relationships:

- **Users** – Stores customer and admin account information
- **Products** – Stores product details
- **Categories** – Stores product category information (many-to-many with Products)
- **Product_Variants** – Stores variant-specific data (color, size, price, stock, SKU)
- **Orders** – Stores order header information (user, date, status, delivery mode, payment method)
- **Order_Items** – Stores individual items within each order (variant, quantity, price)
- **Payments** – Stores payment records linked to orders
- **Delivery** – Stores delivery details and estimated times
- **Cart** – Stores active shopping cart items for registered users
- **Audit_Log** – Stores administrative action logs

B.2 Workflow Diagram



The workflow diagram illustrates the end-to-end order placement process:

1. Customer browses products and adds items to cart
2. Customer initiates checkout
3. System validates stock for all cart items
4. Customer selects delivery mode and payment method
5. System processes payment
6. On success: System confirms order and atomically deducts stock
7. On failure: System cancels the order with stock unchanged
8. System sends order confirmation notification

Appendix C: To Be Determined List

The items listed below remain open and will be resolved during the implementation or pre-deployment phases of the project:

1. A definitive choice between Node.js (Express.js) and Java (Spring Boot) for the server-side framework has not yet been locked in; this decision hinges on a team skills assessment.
2. Whether the payment processing module will interface with a live third-party gateway (such as Stripe or Razorpay) or operate using simulated mock transactions for the academic submission is still under discussion.
3. The client-side rendering library (React.js versus Vue.js) will be chosen following an internal evaluation of team proficiency and long-term maintainability considerations.
4. The transactional email provider (options include SendGrid, Mailgun, or a self-hosted SMTP relay) for dispatching order confirmations and status alerts has not been finalized.
5. A potential future phase involving automated shipment tracking through third-party logistics APIs has been conceptually scoped but is not committed for the initial product launch.
6. The precise classification of Texas municipalities into "major metro" versus "regional" tiers for the delivery estimation algorithm requires formal input from BrightBuy's operations team.
7. The underlying technology powering the AI Chat Assistant (candidate options: OpenAI API, Google Dialogflow, or a custom rule-based engine) is pending a feasibility review.

Appendix D: Team Members

Name	Index Number
Jayawardena R.T.K	240295V
Gamage K.G.T.S.S	240183C
Gunarathna I.K.L.	240195N
Nishaya K.A.R.	240470E
Munasinghe B. S. M. P	240439T